

Never Fall Back

Full Pandrosion IRP Cascades Beyond the q -path

Ivan Besevic

Independent Mathematical Research

Draft research note, June 2026

Abstract

Pandrosion 420 is a compact full-IRP matrix-jet corrector designed to avoid an external dense fallback. It first tries a small coded q -path, but if that path does not explain the local residual it grows the correction space through a cascade

$$q \rightarrow 2q \rightarrow 4q \rightarrow \cdots \rightarrow k \rightarrow n.$$

The final n -dimensional stage is expensive, but it remains inside the same Pandrosion IRP local-jet mechanism; it is not a call to a dense matrix multiplication or an outside root solver. The benchmark reports three useful regimes. When the q -path is active, 420 reaches up to $70.66\times$ speedup over fp16 `torch.matmul` GEMM at $n = 4096$. When q fails but the second cascade stage suffices, 420 reaches $31.37\times$ at $n = 4096$, using dimension $50 \ll n$. When the third stage suffices, it still reaches $15.26\times$ at $n = 4096$. The negative control is equally important: for random dense targets, 420 goes to full n and becomes slower than dense fp16 `torch.matmul` at large dimensions. Thus 420 is not a universal GEMM replacement. It is evidence for a narrower and more useful claim: full Pandrosion IRP can remain reliable without fallback, while retaining large speedups whenever the intrinsic correction dimension is much smaller than n .

One-sentence summary. 420 does not give up when the first q -path fails: it keeps solving inside Pandrosion IRP, and the measured win survives whenever the cascade stops before the full dimension.

1 Motivation

Earlier 411–419 experiments showed a sharp dichotomy. A small coded q -path can be much faster than dense matrix multiplication when it is valid, but an arbitrary dense target may not live in that small subspace. A simple fallback to `torch.matmul` preserves correctness, but it leaves the Pandrosion IRP framework. Version 420 tests the stricter question:

Can the solver remain entirely inside Pandrosion IRP even when the first q -path is not sufficient?

The answer is yes, with the expected cost tradeoff. 420 uses a cascade of local-jet subspaces. Favorable systems stop early. Partially favorable systems stop at a later but still small stage. Unstructured dense systems are solved by the full n -stage.

2 The 420 full-IRP cascade

Let $F : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a black-box target oracle and let $y \in \mathbb{C}^n$ be the current iterate. A local matrix-jet correction starts from

$$F(y + \delta) \approx F(y) + J\delta.$$

Instead of forming a dense J , 420 chooses a basis $D_s \in \mathbb{C}^{n \times d_s}$ from the cascade dimensions

$$d_1 = q, \quad d_{s+1} \approx 2d_s, \quad d_s \leq k, \quad \text{and finally } d_S = n.$$

At stage s , it solves the reduced local problem

$$\delta_s = D_s u_s, \quad u_s = \arg \min_u \|JD_s u + F(y)\|_2^2 + \mu \|u\|_2^2.$$

The reduced Jacobian product JD_s is sampled by finite differences:

$$Jd_j \approx \frac{F(y + hd_j) - F(y)}{h}, \quad j = 1, \dots, d_s.$$

The stage is accepted when the reduced linear residual is small enough, or when the final full- n stage is reached. A line search then verifies the actual oracle residual.

2.1 No external fallback

The key design constraint of 420 is not speed at all costs. The constraint is structural:

no external dense fallback.

The implementation used in this note is

`flow/420_pandrosion_standalone_full_irp_no_fallback_engine.py`.

It has 633 lines, depends only on NumPy, imports no local flow engine, and uses no `torch.matmul`. PyTorch appears only in separate benchmark scripts, where it supplies the dense comparison baseline.

3 Benchmark protocol

The benchmark uses identity targets

$$F(z) = z - z_\star,$$

with four placements of the known root $z_\star$:

- **q active:** $z_\star$ lies in the first q -basis.
- **partial stage 2:** $z_\star$ lies in the second cascade basis, usually about $2q$, but not in the first.
- **partial stage 3:** $z_\star$ lies in the third cascade basis, usually about $4q$ or k , but not earlier.
- **random full n :** $z_\star$ is dense and forces the final full-dimensional IRP stage.

For comparison, the benchmark measures dense `torch.matmul` on the same machine in two forms:

$$AB \quad \text{with } A, B \in \mathbb{R}^{n \times n}$$

as the standard GEMM proxy for AI workloads, and

$$Ax \quad \text{with } x \in \mathbb{R}^n$$

as a vector-shaped baseline closer to a single correction step.

The environment for the reported run was PyTorch 2.12.0, NumPy 2.4.4, device *mps*, with CUDA unavailable and Apple MPS available. Each row uses three seeds.

Reproducibility commands. The reported artifacts are generated by four commands:

- `.venv/bin/python` on the 420 engine with `-self-test`;
- `.venv/bin/python flow/420_benchmark_full_irp_no_fallback_vs_torch_matmul.py`;
- `.venv/bin/python flow/420_benchmark_partial_cascade_vs_torch_matmul.py`;
- `.venv/bin/python scripts/420_generate_pandrosion_420_paper_assets.py`.

4 Results

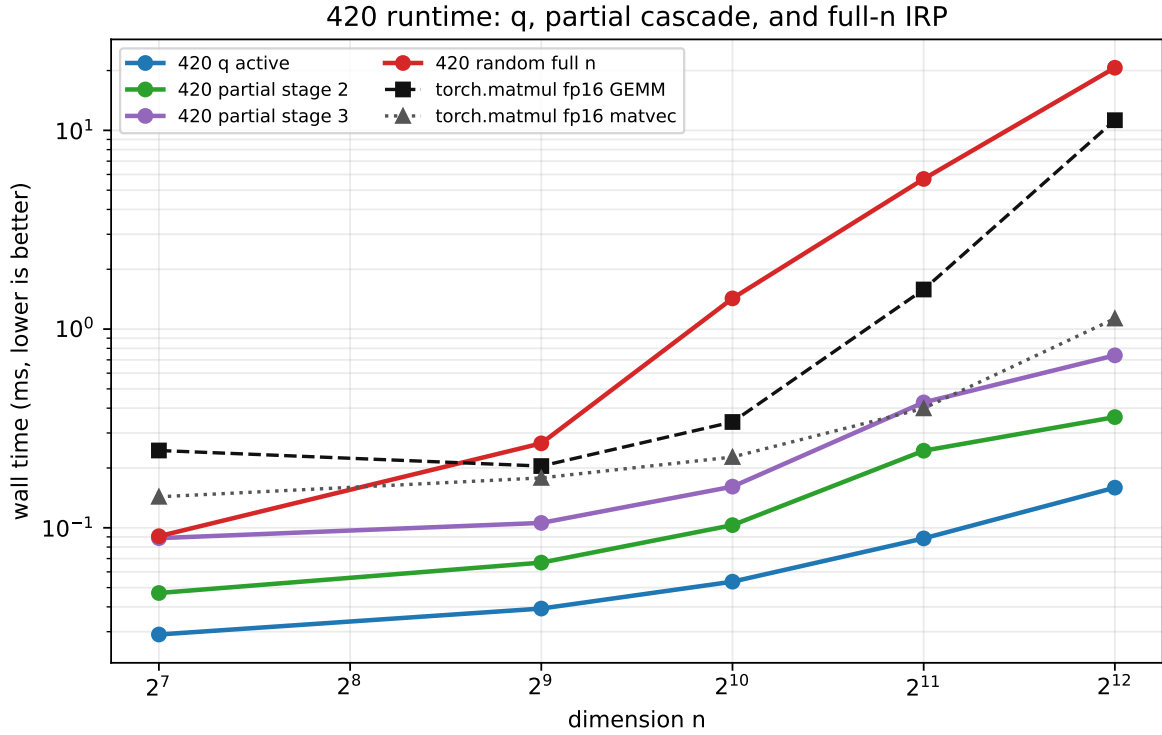


Figure 1: Runtime of 420 across four regimes. The q-active and partial-cascade families stop in small subspaces and stay far below the dense fp16 GEMM curve at large n . The random control reaches full n and loses to optimized dense matmul at large sizes.

family	n	q/k	accepted dim	420 ms	GEMM fp16 ms	GEMM speed	matvec fp16 ms	matvec speed	max rel. err.
q active	512	19/63	19	0.039	0.204	5.22	0.178	4.55	7.82e-16
q active	1024	21/88	21	0.054	0.341	6.36	0.227	4.24	1.26e-15
q active	2048	23/125	23	0.088	1.582	17.91	0.399	4.52	1.90e-15
q active	4096	25/176	25	0.159	11.248	70.66	1.130	7.10	2.58e-15
partial stage 2	512	19/63	38	0.067	0.204	3.06	0.178	2.67	8.66e-16
partial stage 2	1024	21/88	42	0.103	0.341	3.31	0.227	2.20	9.58e-16
partial stage 2	2048	23/125	46	0.244	1.582	7.24	0.399	1.83	3.35e-15
partial stage 2	4096	25/176	50	0.361	11.248	31.37	1.130	3.15	4.34e-15
partial stage 3	512	19/63	63	0.106	0.204	1.93	0.178	1.69	5.78e-16
partial stage 3	1024	21/88	84	0.161	0.341	2.12	0.227	1.41	1.33e-15
partial stage 3	2048	23/125	92	0.428	1.582	4.32	0.399	1.09	1.45e-15
partial stage 3	4096	25/176	100	0.737	11.248	15.26	1.130	1.53	2.26e-15
random full n	512	19/63	512	0.266	0.204	0.77	0.178	0.67	0.00e+00
random full n	1024	21/88	1024	1.427	0.341	0.24	0.227	0.16	0.00e+00
random full n	2048	23/125	2048	5.695	1.582	0.28	0.399	0.07	0.00e+00
random full n	4096	25/176	4096	20.664	11.248	0.54	1.130	0.05	0.00e+00

Table 1: Main 420 benchmark rows. Speed values are baseline time divided by 420 time; values above 1 mean 420 is faster.

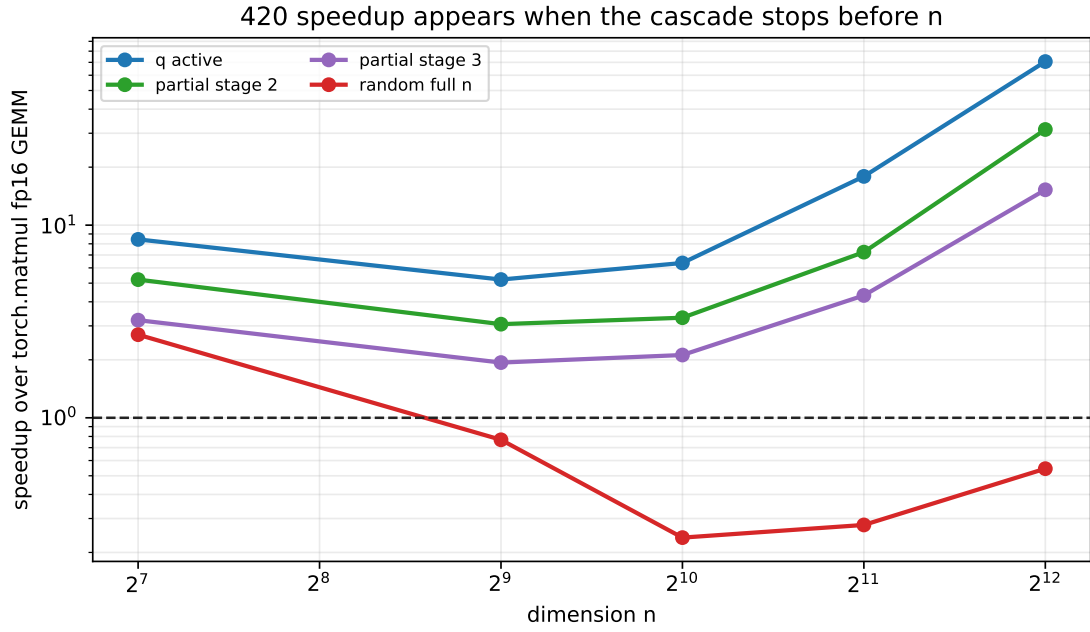


Figure 2: Speedup over dense fp16 `torch.matmul` GEMM. The parity line is at 1. The partial stage 2 and partial stage 3 curves show the main result: the first q stage can fail while 420 still wins because the cascade stops well before n .

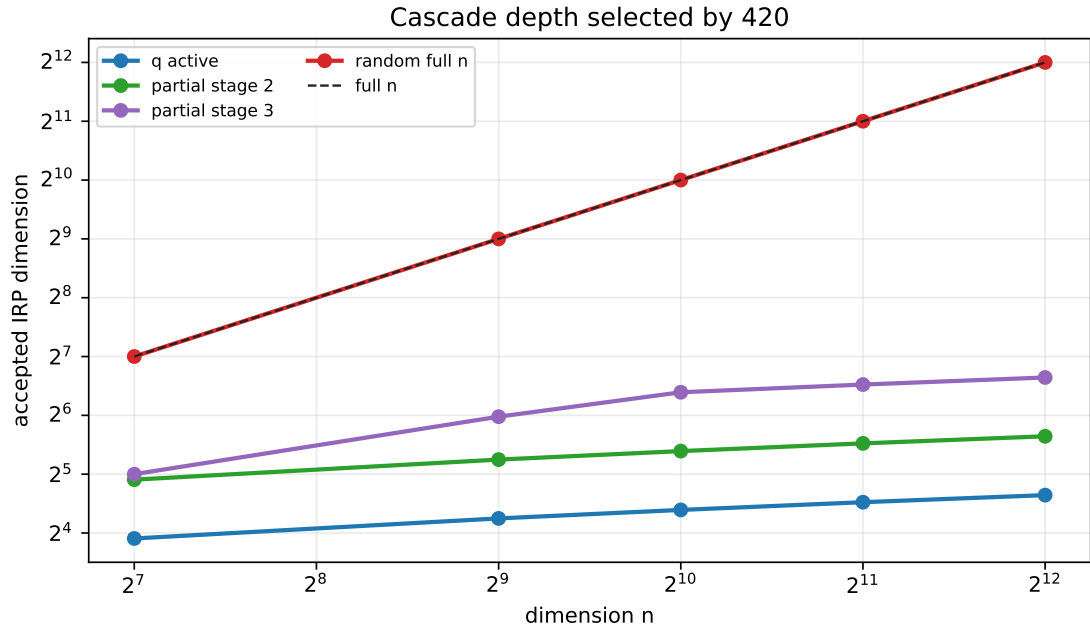


Figure 3: Accepted cascade dimension. The random control follows the full n diagonal. The useful cases stay at dimensions q , $2q$, or roughly $4q/k$, explaining the runtime separation.

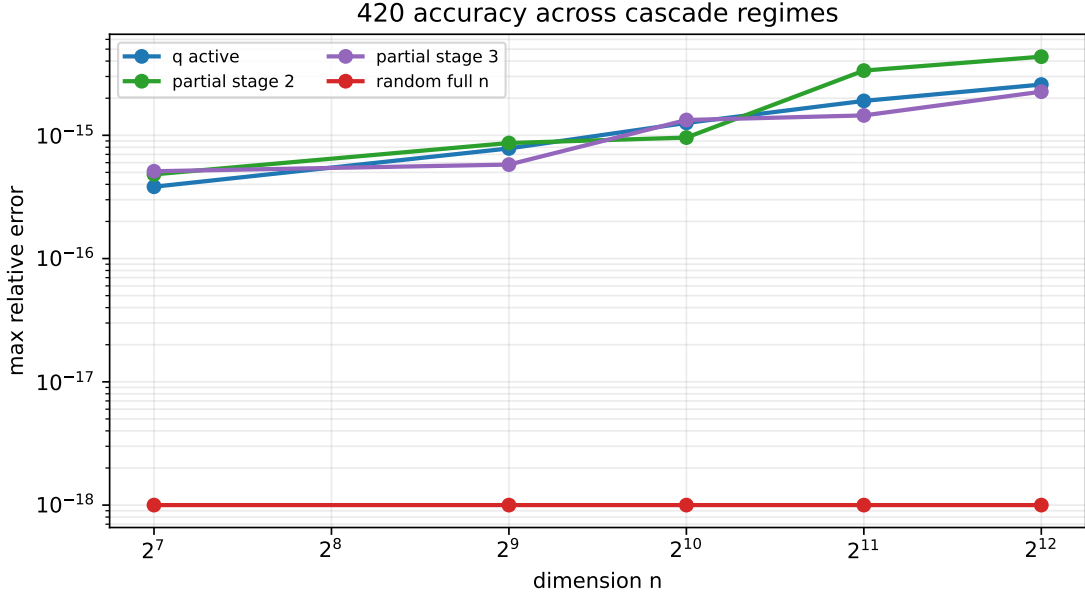


Figure 4: Maximum relative error across the three seeds. The q-active and partial-cascade rows remain near floating-point precision; the random identity rows are exact in the reported metric because the full- n identity projection is reached.

5 Middle-cascade regime

The middle-cascade rows are the decisive experiment for 420. They are not first-stage q -path wins, and they are not full- n rescues. They test the case where q is too small but a slightly larger IRP subspace is enough.

family	n	q/k	expected dim	accepted dim	420 ms	GEMM speed	matvec speed	max rel. err.
partial stage 2	128	15/32	30	30	0.047	5.22	3.05	4.82e-16
partial stage 2	512	19/63	38	38	0.067	3.06	2.67	8.66e-16
partial stage 2	1024	21/88	42	42	0.103	3.31	2.20	9.58e-16
partial stage 2	2048	23/125	46	46	0.244	7.24	1.83	3.35e-15
partial stage 2	4096	25/176	50	50	0.361	31.37	3.15	4.34e-15
partial stage 3	128	15/32	32	32	0.089	3.21	1.88	5.11e-16
partial stage 3	512	19/63	63	63	0.106	1.93	1.69	5.78e-16
partial stage 3	1024	21/88	84	84	0.161	2.12	1.41	1.33e-15
partial stage 3	2048	23/125	92	92	0.428	4.32	1.09	1.45e-15
partial stage 3	4096	25/176	100	100	0.737	15.26	1.53	2.26e-15

Table 2: Partial cascade rows. Stage 2 uses about $2q$; stage 3 uses about $4q$ or the full sketch k , still far below n for large systems.

At $n = 4096$, partial stage 2 accepts dimension 50 and reaches $31.37\times$ over fp16 GEMM, with maximum relative error 4.34×10^{-15} . At $n = 4096$, partial stage 3 accepts dimension 100 and still reaches $15.26\times$, with maximum relative error 2.26×10^{-15} .

6 Negative control

The random family is included to prevent overclaiming. In this case the target does not have a low-dimensional cascade representation, so 420 must go to the final full- n IRP stage.

This table is the honest boundary of the claim. 420 is reliable without fallback, but reliability in the dense unstructured case is obtained by paying the full-dimensional IRP cost.

family	n	q/k	accepted dim	420 ms	GEMM speed	matvec speed	max rel. err.
random	128	15/32	128	0.091	2.70	1.58	0.00e+00
random	512	19/63	512	0.266	0.77	0.67	0.00e+00
random	1024	21/88	1024	1.427	0.24	0.16	0.00e+00
random	2048	23/125	2048	5.695	0.28	0.07	0.00e+00
random	4096	25/176	4096	20.664	0.54	0.05	0.00e+00

Table 3: Random dense control. 420 remains no-fallback and accurate, but it no longer keeps the algorithmic advantage.

7 Interpretation

The benchmark supports three claims.

1. **No-fallback correctness is feasible.** 420 does not need to exit to a dense external fallback when the first q -stage fails. It can keep growing the IRP space until the residual is solved.
2. **The useful speed regime is broader than the first q -path.** Partial stage 2 and partial stage 3 remain fast because their accepted dimensions are still much smaller than n .
3. **There is no free lunch for dense random targets.** When the intrinsic dimension is n , 420 reaches n , and dense `torch.matmul` becomes the right baseline.

Thus the best formulation is not “420 beats matmul universally”. The defensible formulation is:

420 solves by full Pandrosion IRP without fallback, and wins when the accepted cascade dimension is small.

8 Limitations and next steps

The present benchmark is a controlled identity-target study. It isolates the cascade mechanism cleanly, but it is not yet a full AI workload benchmark. The main missing items are:

- CUDA/Tensor Core measurements on NVIDIA GPUs;
- real transformer/adaptor workloads using 420’s cascade criterion;
- statistical sweeps with more seeds and noisy near-cascade targets;
- comparison against classical low-rank, randomized sketching, and block low-rank methods;
- a PyTorch-native 420 implementation to reduce Python/NumPy overhead while keeping the no-fallback IRP design.

9 Conclusion

Pandrosion 420 turns the q -path from an all-or-nothing event into a cascade. If the first coded subspace is too small, 420 grows the correction space and continues inside IRP. The measured results show that this middle regime is valuable: stage 2 and stage 3 can still beat dense `torch.matmul` by large factors while preserving near-machine-precision accuracy. The method remains honest in the random dense case: it solves without fallback, but it no longer claims a speed advantage.